Open-source code check

Name: Vincent Yang
Assignment: Assignment 1: Reinforcement Learning
Repository: https://github.com/vi-yang-NU/RL
Date of the check: October 30, 2026

The rule

> You are not allowed to use open source code that does not have standard python packages. In other words, you can only use modules that can be installed through pip/conda in your virtual environment.

Result

The code follows the rule. Every module our Python code imports is either part of the Python standard library, our own code, or a package installed with pip from a package index. No third-party code is copied into the repository.

The check is a script in the repository, so it can be repeated with one command from the project folder:

```
python scripts/check_imports.py
```

It prints the tables below and ends with PASS on both machines the assignment ran on.

Environment	Python	Third-party modules checked	Verdict
Laptop, both requirements files	3.9.6	12 of 12	PASS
deepdish2 GPU server, requirements.txt only	3.10.12	8 of 8 installed there	PASS

The laptop runs the dashboard, builds the report and did the CPU runs. deepdish2 did the GPU training runs. The four modules not installed on deepdish2 belong to the dashboard and the report builder, which only run on the laptop, where they pass.

What the check does

1. **Every import.** All 64 Python files are parsed with Python's `ast` module, and each imported module is classified as standard library, project code or third-party. A third-party module must be installed in the virtual environment's site-packages. Its pip package name, version, installer and install source are read from the package's own metadata.
2. **Where each package came from.** pip records a `direct_url.json` file for anything installed from a git repository, a URL or a local folder. None of the packages has one, so every package came from a package index.
3. **The requirements files.** Every third-party package is either listed in requirements.txt or requirements-ui.txt, or required by a package that is. The files contain package names only:

no git URLs, local paths, editable installs or extra indexes.

4. **Copied code.** All 70 source files (.py, .js, .html, .sh, .css) apart from the check script itself are searched for third-party license headers, "adapted from" or "copied from" notes, scripts or stylesheets loaded from outside the repository, and distinctive lines of widely copied REINFORCE code (Karpathy's pg-pong, the PyTorch REINFORCE example, OpenAI Spinning Up, Stable-Baselines). Nothing was found.

Third-party packages

Module	pip package	How it is installed	Used for	Version: laptop · deepdish2
numpy	numpy	listed in requirements.txt	arrays throughout	2.0.2 · 2.0.2
torch	torch	listed in requirements.txt	the policy and value networks, training	2.8.0 · 2.14.1+cu126
gymnasium	gymnasium	listed in requirements.txt	the CartPole-v1 and Pong-v5 environments	1.1.1 · 1.4.0
ale_py	ale-py	listed in requirements.txt	the Atari emulator behind Pong-v5	0.11.2 · 0.12.1
pygame	pygame (laptop), pygame-ce (deepdish2)	dependency of gymnasium[classic-control]	rendering CartPole frames	2.6.1 · 2.5.8
numba	numba	listed in requirements.txt	the optional CUDA kernels	0.60.0 · 0.60.0
matplotlib	matplotlib	listed in requirements.txt	every figure	3.9.4 · 3.10.9
PIL	pillow	dependency of matplotlib	frame images in the game viewer, table images	11.3.0 · 12.3.0
streamlit	streamlit	listed in requirements-ui.txt	the dashboard	1.50.0 · not installed
pandas	pandas	listed in requirements-ui.txt	tables in the dashboard	2.3.3 · not installed
reportlab	reportlab	listed in requirements-ui.txt	building the PDF report	4.4.4 · not installed
tornado	tornado	dependency of streamlit	the script that screenshots the dashboard for one report figure	6.5.10 · not installed

- On deepdish2, torch is the CUDA build from PyTorch's official pip index. The setup script installs it with `pip install torch --index-url https://download.pytorch.org/whl/cu126`. Every other package comes from PyPI.
- requirements.txt also lists two pip packages, on Linux only, that Numba needs to compile the CUDA kernels on the GPU server. They are `nvidia-cuda-nvcc-cu12`, NVIDIA's compiler library, and `cuda-python`, NVIDIA's Python bindings. Our code does not import them

directly.

Standard library and our own code

Standard library modules: `__future__`, argparse, ast, base64, collections, concurrent, contextlib, dataclasses, datetime, fractions, getpass, html, importlib, io, json, math, os, pathlib, platform, random, re, shlex, shutil, socket, subprocess, sys, sysconfig, tempfile, textwrap, time, traceback, typing, unittest, urllib, warnings, webbrowser, xml.

Our own packages: `rl` (the solvers, environments, training and plotting), `report` (the report builder), `ui` (the dashboard), `tests` (the validation harness and unit tests), and the scripts in `scripts/`.

Code provided by the course: the `preprocess` function in rl/pg/envs.py is the handout's function. It is used unchanged, as the assignment requires for Pong frames, and is credited to the handout in the code.

Outside Python

- **Lean proofs, in the lean folder.** These are formal proofs of facts the code relies on. Examples are that the Bellman operator of the shuttle model is a contraction, that the backward recursion computes the discounted returns, and that a baseline leaves the expected policy gradient unchanged. The proofs are ours, but they build on Mathlib, an open-source Lean library that Lean's own package manager downloads, not pip or conda. No Python code imports or runs Lean or Mathlib, and solving, training and evaluating do not need them. One unit test compares the solvers with a file of exact values exported from the Lean model, lean/reference/shuttle_reference.json, and it skips itself when that file is absent.
- **Google Chrome.** scripts/explore_collage.py opens the dashboard in a headless Chrome window to screenshot it for one report figure. Chrome is an installed browser, not a code dependency, and no assignment code uses it.